

TEMA I

DESARROLLO DE APLICACIONES EN PHYTON

*"El talento gana partidos, pero el trabajo en equipo y la inteligencia
ganan campeonatos"*

Michael Jordan

1.1 INTRODUCCIÓN

Breve historia de Python (platzi.com)

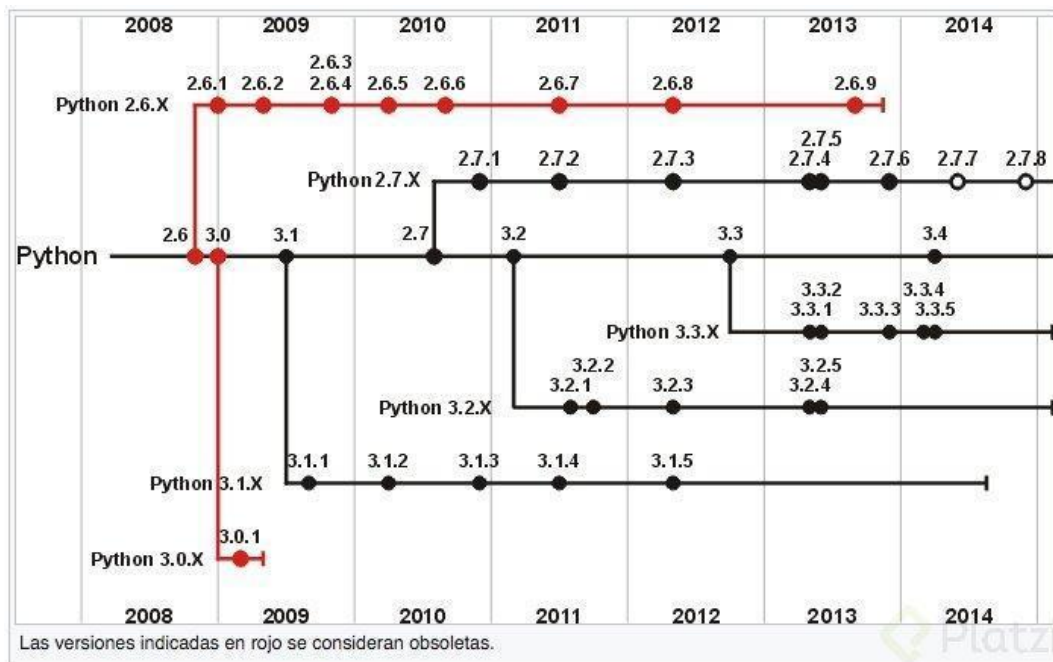
La historia de Python como lenguaje de programación inicia a finales de los 80s y principios de los 90s con Guido Van Rossum, una historia de 29 años de desarrollo.

En una navidad de 1989, Guido Van Rossum, quien trabajaba en el CWI (un centro de investigación holandés), decidió empezar un proyecto como pasatiempo dándole continuidad a ABC, un lenguaje de programación que se desarrolló en el CWI.

ABC fue desarrollado a principios de los 80s como alternativa a BASIC, fue pensado para principiantes por su facilidad de aprendizaje y uso. Su código era compacto pero legible.

El proyecto no trascendió ya que el hardware disponible en la época hacía difícil su uso. Así que Van Rossum le dio una segunda vida creando Python.

A Guido Van Rossum le gustaba mucho el grupo Monty Python, por esta razón escogió el nombre del lenguaje. Actualmente Van Rossum sigue ejerciendo el rol central decidiendo la dirección de Python.



En 1991, Van Rossum publicó el código de la versión 0.9.0 en alt.sources. En esta versión ya teníamos disponibles clases con herencias, manejo de excepciones, funciones y los tipos modulares.

En esta versión aparece un sistema de módulos adoptado de Modula-3, un lenguaje de programación estructurado y modular, el cual Guido describe como una de las mayores unidades de programación de Python. Por ejemplo, el modelo de excepciones de Python es parecido al de Modula-3.

Para 1994 se creó comp.lang.python, un foro de discusión de Python que marcó un hito en su popularidad y multiplicó su cantidad de usuarios.

Versión 1.0

Para este mismo año, Python llega a la versión 1.0 que incluyó herramientas de la programación funcional como lambda, reduce, filter y map.

Herramientas que llegaron al lenguaje gracias a un hacker de Lisp, una familia de lenguajes de programación de computadora de tipo multiparadigma.

La última versión liberada en CWI fue Python 1.2, en 1995, Van Rossum continuó su trabajo en la Corporation for National Research (CNRI) en Virginia, donde lanzó varias versiones del lenguaje.

Para la versión 1.4, vemos nuevas características, muchas inspiradas en Modula-3, y además soporte built-in para los números complejos.

Para el 2000, el equipo principal de desarrolladores de Python se cambió a [BeOpen.com](https://beopen.com) para formar el equipo de BeOpen Python Labs. CNRI pidió que la versión 1.6 fuera publicada al momento en que el equipo abandonó CNRI.

La versión 1.6 publicada en el CNRI incluye una licencia sustancialmente más larga que la de las versiones publicadas en CWI. La nueva licencia incluía una cláusula indicando que ésta se regía por las leyes de Virginia.

La Free Software Foundation (FSF), fundación creada por Stallman con el objetivo de incentivar el Software Libre, argumentó que la cláusula era incompatible con GNU GPL. Así que acordaron cambiar Python a una licencia de Software Libre, que lo haría compatible con GPL.

Por esto, Python 1.6.1 es básicamente lo mismo que 1.6, con un par de arreglos de bugs y una nueva licencia compatible con GPL.

Versión 2.0

Para Python 2.0 se incluyó la generación de listas, una de las características más importantes del lenguaje de programación funcional Haskell. Además, incluyó un sistema de recolección de basura capaz de recolectar referencias cíclicas.

En 2001, se crea la Python Software Foundation, la cual a partir de Python 2.1 es dueña de todo el código, documentación y especificaciones del lenguaje. La fundación se basó en el modelo de la Apache Software Foundation.

Versión 3.0

La última gran actualización de Python fue en 2008 con Python 3.0 con el PEP 3000, diseñado para rectificar fallas fundamentales en el diseño del lenguaje.

“Reduce feature duplication by removing old ways of doing things”.

La filosofía de Python 3.0 es la misma de las versiones anteriores, sin embargo, Python como lenguaje ha acumulado nuevas y redundantes formas de programar una misma tarea.

Python 3.0 ha hecho énfasis en eliminar constructores duplicados y módulos con el objetivo de cumplir la regla de “tener solo un modo obvio de hacer las cosas”.

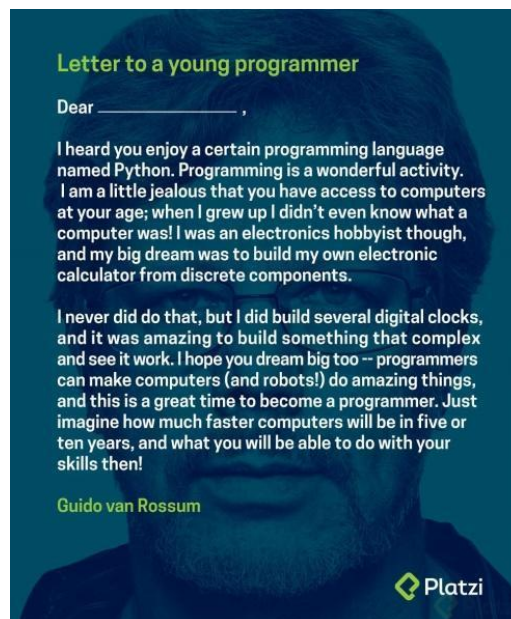
Las versiones de Python 3.x y Python 2.x fueron planeadas para coexistir por varios releases que se lanzaron en paralelo, donde Python 2.6 se lanzó al tiempo con 3.0, incluyendo nuevas características y alertas que resaltan el uso de herramientas eliminadas en la versión 3.0

De igual manera, 2.7 fue lanzado al tiempo con 3.1 e incluye características de la nueva versión, siendo la 2.7 la última publicación en la serie 2.x, la cual actualmente sólo recibe actualizaciones de seguridad y dejará de tener soporte en 2020.

Python 3.0 rompe la compatibilidad hacia atrás del lenguaje, ya que el código de Python 2.x no necesariamente debe correr en Python 3.0 sin modificación alguna.

Uno de los proyectos más impresionantes escritos en Python es el servidor de Dropbox, (lugar donde hoy en día trabaja Guido) que hoy en día sirve a millones de personas. Otro uso increíble es por la comunidad científica como herramienta para Machine Learning.

Por último, un mensaje de Guido Van Rossum para todos los programadores jóvenes, o aquellos que están empezando a programar.



Principales ventajas de Python

- Es un lenguaje de muy alto nivel, es fácil de aprender gracias a que su sintaxis es bastante legible para los humanos.
- Tipado fuerte pero dinámico
- Es orientado a objetos, permite herencia múltiple

- Es de código abierto (certificado por la OSI).
- Es interpretado
- Es un lenguaje maduro (29 años).
- Es fácilmente extensible e integrable en otros lenguajes (C, java).
- Está mantenido por una gran comunidad de desarrolladores y hay multitud de recursos para su aprendizaje.
- Viene preinstalado en la mayoría de sistemas.
- Hay gran cantidad de módulo con muchísimas funcionalidades
- Se pueden programar distintos tipos de aplicaciones: de escritorio, web, scripts...

Acceso a datos

En Python, el acceso a bases de datos se encuentra definido a modo de estándar en las especificaciones de DB-API, que puedes leer en la PEP 249. Esto significa que independientemente de la base de datos que utilicemos, los métodos y procesos de conexión, lectura y escritura de datos, desde Python, siempre serán los mismos, más allá del conector.

En nuestro caso particular, utilizaremos sqlite y postgresql.

También se puede trabajar con mysql. Para ello python dispone del módulo MySQLdb.

A diferencia de los módulos de la librería estándar de Python, MySQLdb debes instalado manualmente ejecutando el siguiente comando:

```
sudo apt-get install python-mysqldb
```